fn make_clamp

Sílvia Casacuberta

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `fn make_clamp` in `mod.rs` at commit 0db9c6036 (up to date).

1 Vetting history

- Pull Request #512

2 Versions of definitions documents

When looking for definitions for terms that appear in this document, the following versions of the definitions documents should be used.

- **Pseudocode definitions document:** This proof file uses the version of the pseudocode definitions document available as of September 6, 2021, which can be found at this link (archived here).
- **Proof definitions document:** This file uses the version of the proof definitions document available as of September 6, 2021, which can be found at this link (archived here).

3 Algorithm implementation

3.1 Pseudocode in Python

We present a simplified Python-like pseudocode of the Rust implementation below. The necessary definitions for the pseudocode can be found at "List of definitions used in the pseudocode".

The use of `code`-style parameters in the preconditions section below (for example, `input_domain`) means that this information should be passed along to the `Transformation` constructor.

Preconditions

To ensure the correctness of the output, we require the following preconditions:

- **User-specified types:**
  - Type `T` must have trait `TotalOrd`.

```
def make_clamp((L, U): (T, T)):
    input_domain = VectorDomain(AllDomain(T))
    output_domain = VectorDomain(IntervalDomain(L, U))
    input_metric = SymmetricDistance()
    output_metric = SymmetricDistance()

```

```
    def stability_map(d_in: u32) -> bool:
        return d_in

    def function(data: Vec[T]) -> Vec[T]:
        def clamp(x: T) -> T:
            return x.total_clamp(L, U)
        return list(map(clamp, data))

    return Transformation(
        input_domain, output_domain, function,
        input_metric, output_metric, stability_relation
    )
```

Postconditions

- Either a valid `Transformation` is returned or an error is returned.

4 Proof

The necessary definitions for the proof can be found at "List of definitions used in the proofs".

4.1 Symmetric distance

Theorem 4.1. For every setting of the input parameters `(L, U)` to `make_clamp` such that the given preconditions hold, `make_clamp` raises an exception (at compile time or run time) or returns a valid transformation with the following properties:

1. (Appropriate output domain). For every element v in `input_domain`, `function`(v) is in `output_domain`.
2. (Domain-metric compatibility). The domain `input_domain` matches one of the possible domains listed in the definition of `input_metric`, and likewise `output_domain` matches one of the possible domains listed in the definition of `output_metric`.
3. (Stability guarantee). For every pair of elements v, w in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for `output_metric`, if v, w are `d_in`-close under `input_metric` and `Map(d_in)` $\leq$ `d_out`, then `function`(v), `function`(w) are `d_out`-close under `output_metric`.

Proof. **(Appropriate output domain).** In the case of `make_clamp`, this corresponds to showing that for every vector v of elements of type `T`, `function`(v) is a vector of elements of type `T` which are contained in the interval `[L, U]`. For that, we need to show two things: first, that `function(v)` has type `Vec[T]`. Second, that they belong to the interval `[L, U]`.

Firstly, that `function`(v) has type `Vec[T]` follows from the assumption that element v is in `input_domain` and from the type signature of `function` in line 10 of the pseudocode (Section 3.1), which takes in an element of type `Vec[T]` and returns an element of type `Vec[T]`. If the Rust code compiles correctly, then the type correctness follows from the definition of the type signature enforced by Rust. Otherwise, the code raises an exception for incorrect input type.

Secondly, we need to show that the vector entries belong to the interval `[L, U]`. For that, it is foremost necessary that `L` $\leq$ `U`. This condition is already checked when declaring `output_domain = VectorDomain(IntervalDomain(L, U))` in line 3 of the pseudocode. This check already exists via the construction of `IntervalDomain`, which returns an error if `L` $>$ `U`. The rest follows from the definition of `clamp` in line 11. According to line 11 in the pseudocode, there are 3 possible cases to consider:

1. `x` $>$ `U`: then `clamp(x)` returns `U`.

2. `x` $\in$ `[L, U]`: then `clamp(x)` returns `x`.

3. `x` $<$ `L`: then `clamp(x)` returns `L`.

In all three cases, the returned value of type `T` is contained in the interval `[L, U]`. Hence, the vector `function`(v) returned in line 13 of the pseudocode is an element of `output_domain`.

Lastly, both `L` and `U` have type `T` by the type signature of `make_clamp`. Both the definition of `IntervalDomain` and that of the `clamp` function (line 11 in the pseudocode, which uses the `min` and `max` functions) require that `T` implements `TotalOrd`, which holds by the preconditions.

(Domain-metric compatibility). For `make_clamp`, both the input and output cases correspond to showing that `VectorDomain(T)` is compatible with the symmetric distance metric. This follows directly from the definition of symmetric distance, as stated in "List of definitions used in the pseudocode".

(Stability guarantee). Throughout the stability guarantee proof, we can assume that `function`(v) and `function`(w) are in the correct output domain, by the *appropriate output domain property* shown above.

Since by assumption `Map(d_in)` $\leq$ `d_out`, by the `make_clamp` stability map (as defined in line 7 in the pseudocode), we have that `d_in` $\leq$ `d_out`. Moreover, v, w are assumed to be `d_in`-close. By the definition of the symmetric difference metric, this is equivalent to stating that $d_{Sym}(v, w) = |\text{MultiSet}(v)\Delta\text{MultiSet}(w)| \leq$ `d_in`.

Let $\mathcal{X}$ be the domain of all elements of type `T`. By applying the histogram notation,[1] it follows that

$$d_{Sym}(v, w) = \|h_v - h_w\|_1 = \sum_{z \in \mathcal{X}} |h_v(z) - h_w(z)| \leq \texttt{d_in} \leq \texttt{d_out}.$$

By Definition 3.10 in "List of definitions used in the proofs" and the definition of `clamp` in lines 11–13 in the pseudocode, it follows that the `function` defined in `make_clamp`, which maps elements from `VectorDomain` to `VectorDomain`, is a row transform. Therefore, by Lemma 3.13 in "List of definitions used in the proofs", it follows that for every pair of elements v, w in `input_domain`,

$$d_{Sym}(\texttt{function}(v), \texttt{function}(w)) \leq d_{Sym}(v, w).$$

Then, by the initial assumptions `relation(d_in, d_out) = True` and `d_in` $\leq$ `d_out`, it follows that

$$d_{Sym}(\texttt{function}(v), \texttt{function}(w)) \leq d_{Sym}(v, w) \leq \texttt{d_in} \leq \texttt{d_out}.$$

Hence,

$$d_{Sym}(\texttt{function}(v), \texttt{function}(w)) \leq \texttt{d_out},$$

as we wanted to show. □

[1] Note that there is a bijection between multisets and histograms, which is why the proof can be carried out with either notion. For further details, please consult https://www.overleaf.com/project/60d214e390b337703d200982.